

Kernels and Random Feature Maps

Giammarco Vadacchino

May 2026

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1 Introduction

This project investigates different strategies for extending linear Perceptron models to non-linear classification tasks. In particular, i compare exact kernel methods, implemented through Kernel SVMs, with scalable approximation techniques based on Random Fourier Features (RFF) and Random Binning Features (RB). The objective of the study is to analyze the trade-off between predictive performance, computational efficiency, and scalability. Through experiments on both synthetic and real datasets, i evaluate how well approximate kernel methods can reproduce the behavior of exact kernels while reducing computational complexity.

2 Models

2.1 Linear Perceptron

The Linear Perceptron is one of the simplest supervised learning algorithms for binary classification. Its main objective is to learn a linear decision boundary capable of separating two classes in the input feature space. Given an input sample:

$$x \in \mathcal{R}^d$$

the model computes a linear score:

$$f(x) = w^T x + b$$

where $w \in \mathcal{R}^d$ is the weight vector and $b \in \mathcal{R}$ is the bias term. The predicted class is obtained by applying a threshold to the score:

$$\hat{y} = \begin{cases} 1 & \text{if } f(x) \geq 0 \\ 0 & \text{if } < 0 \end{cases}$$

In the implementation, the original labels are converted from $y \in \{0, 1\}$ to the signed representation $y \in \{-1, 1\}$. This transformation simplifies the training rule, since the correctness of a prediction can be checked through the margin:

$$m_i = y_i f(x_i)$$

if $m_i > 0$ the sample is correctly classified, if instead $m_i \leq 0$ the sample is misclassified or lies exactly on the decision boundary, and updates its parameters. The update rule is:

$$\begin{aligned} w &\leftarrow w + \eta y_i x_i \\ b &\leftarrow b + \eta y_i \end{aligned}$$

where η is the learning rate.

2.2 Perceptron with Random Fourier Features (RFF)

The Perceptron with Random Fourier Features extends the standard Linear Perceptron by introducing an explicit non-linear feature transformation before training. The goal is to allow a linear classifier to operate in a higher-dimensional feature space, where the data may become more easily separable. This method is based on the idea of approximating the Radial Basis Function kernel, also known as the Gaussian kernel:

$$K(x, z) = e^{-\gamma \|x - z\|^2}$$

Instead of computing the kernel matrix explicitly, Random Fourier Features construct a finite dimensional mapping $\phi(x) \in \mathcal{R}^D$ such that:

$$\phi(x)^T \phi(x) \approx K(x, x)$$

In the implementation, the mapping is defined as:

$$\phi(x) = \sqrt{\frac{2}{D}} \cos(Wx + b)$$

where $W \in \mathcal{R}^{D \times d}$ is a random projection matrix sampled from a Gaussian distribution $W \sim \mathcal{N}(0, 2\gamma)$ and $b \in \mathcal{R}^D$ is a random vector sampled uniformly from $b \sim \mathcal{U}(0, 2\pi)$. The parameter D represents the number of random features. After applying this transformation, the Perceptron is trained exactly as a linear model, but on the transformed data:

$$f(x) = w^T \phi(x) + b$$

2.3 Perceptron with Random Binning Features

The Perceptron with Random Binning Features is another non-linear extension of the standard Perceptron. This method is particularly important because it provides an explicit approximation of the Laplace Kernel defined as:

$$K(x, z) = e^{-\gamma \|x - z\|_1}$$

The key idea is to randomly partition the input space into bins. The random binning process induces a partitioning of the input space such that two points are more likely to fall into the same bin if they are close according to the L_1 distance. As a result, the inner product between transformed feature vectors approximates the Laplace kernel:

$$\phi(x)^T \phi(x) \approx K(x, x)$$

For each random partition, a bin is sampled from an exponential distribution:

$$\delta \sim \text{Exp}(\gamma)$$

In the implementation, very small values of δ are avoided by enforcing a minimum threshold. This prevents numerical instability when dividing by very small bin widths. An offset is then sampled uniformly:

$$u \sim \mathcal{U}(0, \delta)$$

Each input sample is assigned to a bin using:

$$\text{bin}(x) = \lfloor \frac{x+u}{\delta} \rfloor$$

Since the binning process is performed across multiple dimensions, the resulting bin indices are combined using a hashing mechanism. The hashed value determines which bucket is activated for each random feature. The resulting representation is a sparse vector:

$$\phi(x) \in \mathcal{R}^{D \cdot B}$$

where D is the number of random binning transformations, and B is the number of buckets. For each transformation, only one bucket is activated. Therefore, the feature vector contains many zeros and relatively few non-zero values. This sparsity can make the representation efficient, especially when compared to dense transformations. The transformed vector is normalized as:

$$\phi(x) = \frac{1}{\sqrt{D}} \phi(x)$$

After the transformation, the perceptron is trained in the same way before:

$$f(x) = w^T \phi(x) + b$$

2.4 Kernel Support Vector Machine (SVM)

The Kernel Support Vector Machine (SVM) is a non-linear classification model based on the principle of margin maximization. Its objective is to find a decision boundary that not only separates the training data but also maximizes the margin between the two classes, which typically leads to better generalization. In its kernelized form, the decision function is:

$$f(x) = \sum_{i=1}^n a_i y_i K(x_i, x) + b$$

where a_i are the dual coefficients, $y_i \in \{-1, 1\}$ are the labels, and $K(x_i, x)$ is the kernel function. The model is trained by solving the dual optimization problem:

$$\max_a \sum_{i=1}^n a_i - \frac{1}{2} \sum_{i,j} a_i a_j y_i y_j K(x_i, x_j)$$

subject to:

$$0 \leq a_i \leq C, \sum_i a_i y_i = 0$$

The parameter C controls the trade-off between margin maximization and classification errors. In this project i used both Laplace and Gaussian kernels. The dual variables are optimized iteratively, and the bias b is computed using support vectors with $0 < a_i < C$. Compared to Perceptron-based models, the Kernel SVM is more expressive and typically achieves better performance. However, it is also computationally more expensive due to the need to compute the kernel matrix $K \in \mathcal{R}^{n \times n}$.

3 Methodology

3.1 Dataset and Preprocessing

In this project i use a synthetic dataset and a real-world dataset for binary classification in order to evaluate model performance under different conditions. The synthetic dataset is generated using the `make_classification` function from `sklearn.datasets` and consists of 2.5K samples with 10 features, including 4 informative and 3 redundant ones. A label noise of 5% (`flip_y = 0.05`) is introduced, and the class separation is set to 0.8, making the problem moderately challenging and not perfectly separable. The real-world dataset (UCI Credit Card Dataset, last access: 6/05/2026) originally contains approximately 30K samples and 23 features. In order to maintain a fair comparison with the synthetic dataset and reduce computational cost, a random subset of 2.5K samples is selected for the experiments. This dataset is included to assess model performance in a more practical and realistic scenario.

Both datasets are split into training (70%) and test (30%) sets using a fixed random seed for reproducibility. Features are then standardized ensuring zero mean and unit variance. This step improves convergence and stability, especially for models sensitive to feature scaling.

3.2 Hyperparameter Tuning

Hyperparameters are selected differently for the Perceptron based models and the kernel SVM. For the kernel SVM, hyperparameter tuning is performed using k fold cross validation (with $k = 5$). The parameters explored are:

- Regularization parameter: $C \in \{\text{np.logspace}(-2, 2, 6)\}$
- Kernel parameter: $\gamma \in \{\text{np.logspace}(-2, 1, 6)\}$

For each combination of C and γ the model is trained on the training folds and evaluated on the validation fold. For the Perceptrons with Random Features the kernel bandwidth parameter γ is not tuned independently. Instead, the best value γ obtained from the corresponding Kernel SVM is reused. This choice ensures consistency between exact and approximate kernel methods.

3.3 Experimental Protocol for Random Features

Since Random Fourier Features and Random Binning Features involve stochastic transformations, their performance may vary depending on the random initialization. To account for this variability, each experiment is repeated multiple times (5 runs) using different random seeds. For each run:

- The feature mapping is regenerated
- The model is trained and evaluated independently

The final results are obtained by aggregating the outcomes across runs, computing both the mean and the standard deviation. This provides a more reliable estimate of model performance and stability.

4 Model Comparison

4.1 Test Error vs Number of Features

The figure 1 shows how the number of random features D affects the performance of approximate kernel methods on both synthetic and real datasets.

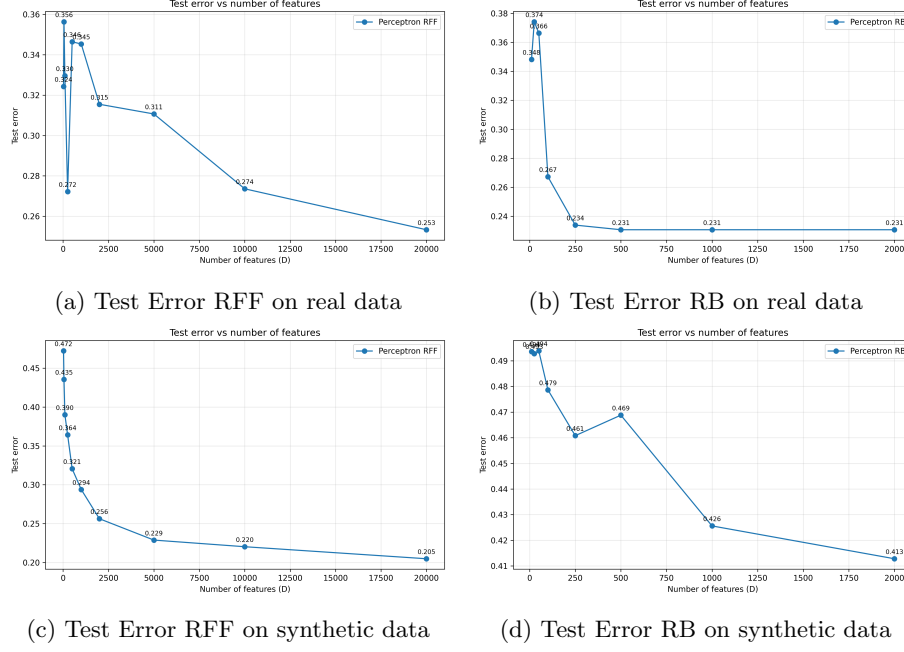


Figure 1: Test Error Comparison

Overall, the results show that increasing the number of random features improves performance up to a saturation point, after which the gain becomes minimal. Among the two methods, Random Fourier Features provide a more stable and scalable approximation, while Random Binning offers faster improvements at lower dimensions.

4.2 Kernel Approximation Comparison

The figure 2 shows the effectiveness of Random Fourier Features and Random Binning Features as approximate kernel methods. Their performance is compared against the exact Kernel SVM and the linear Perceptron baseline by varying the number of random features D .

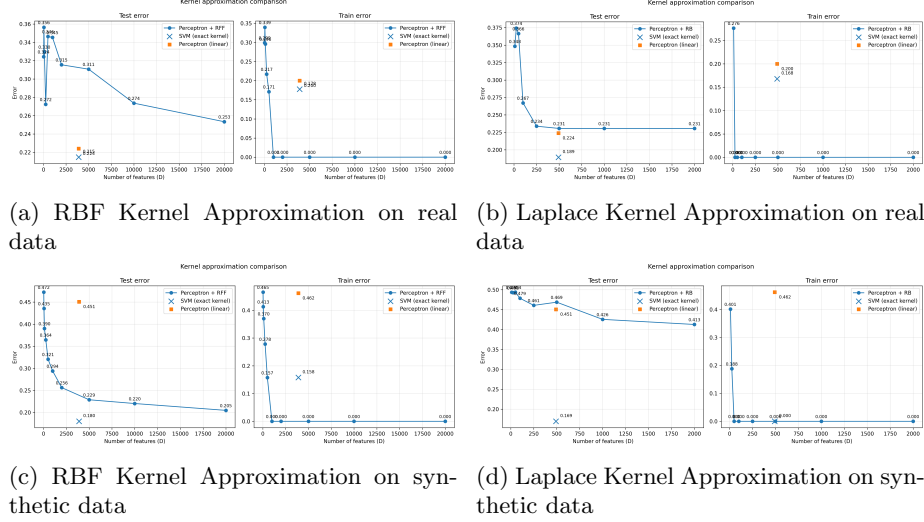


Figure 2: Kernel Approximation Comparison

The results show that increasing the number of random features generally improves the approximation quality of both RFF and RB. In particular, RFF achieves more stable improvements and approaches the performance of the exact Kernel SVM for large values of D . RB also benefits from larger feature spaces, although its performance is generally less stable and remains farther from the exact kernel solution. Additionally, training errors decrease rapidly as the feature dimension increases, showing the higher expressive power of the transformed representations.

4.3 Effect of Kernel Bandwidth (γ)

The figure 3 shows the effect of the kernel bandwidth parameter γ on training and test error for exact kernel SVMs and approximate kernel methods based on RFF and RB features. The analysis highlights how the bandwidth parameter influences approximation quality and generalization performance for both Laplace and RBF kernels.

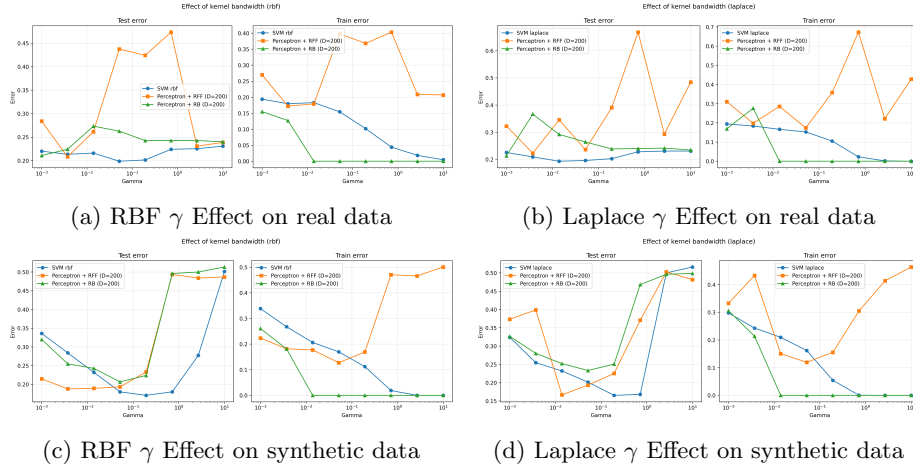


Figure 3: Effect Bandwidth Comparison

The results show that the bandwidth parameter γ has a strong impact on both exact and approximate kernel methods. For moderate values of γ , the exact SVM generally achieves the lowest test error, confirming the effectiveness of kernel learning when the bandwidth is properly tuned. Among the approximation methods, RFF tends to provide more stable and competitive results, especially with the RBF kernel, while RB shows greater variability depending on the dataset and the kernel type. In several cases, very large values of γ lead to overfitting, as shown by near-zero training error combined with increasing test error.

4.4 Runtime vs Accuracy Trade-off

The figure 4 shows the trade-off between runtime and accuracy for exact kernel SVMs and approximate kernel methods. By varying the number of random features D , i analyze how approximation quality improves with computational cost.

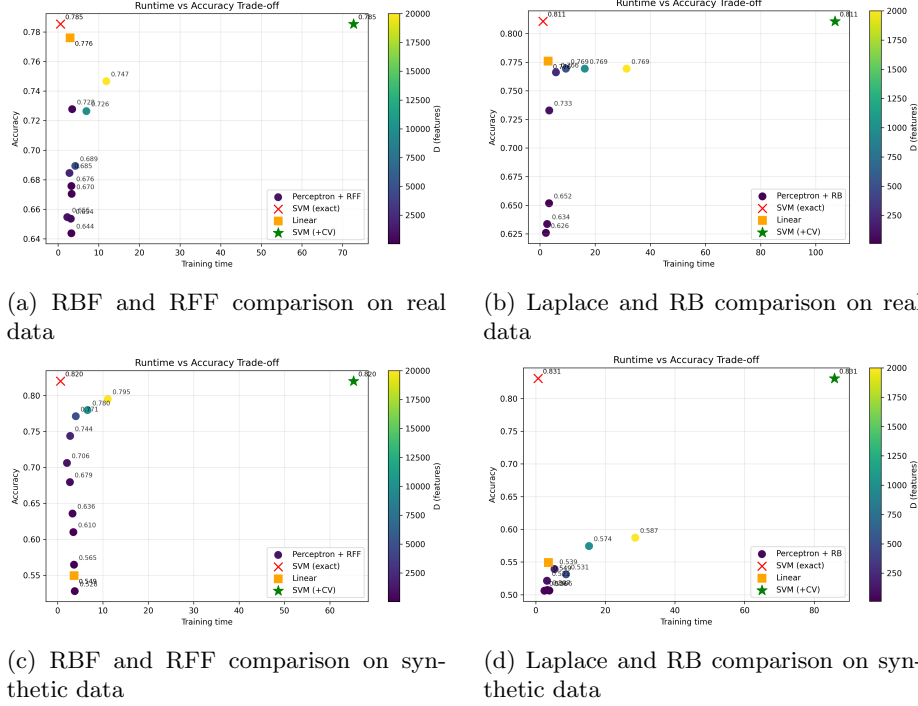


Figure 4: Runtime vs Accuracy Comparison

The plots show the trade-off between runtime and accuracy. Exact kernel SVMs achieve the best performance but require higher computational cost (if we consider that the best practice is to tune the hyperparameters). Approximate methods significantly reduce training time while maintaining competitive accuracy, especially when increasing the number of random features D . In general, RFF provides the most stable balance between efficiency and predictive performance.

5 Conclusion

The results showed that kernel methods significantly outperform the standard linear Perceptron on non-linearly separable synthetic datasets, confirming the effectiveness of kernel learning in capturing complex decision boundaries. Exact kernel SVMs achieved the best overall accuracy, although at a higher computational cost.

Among the approximation methods, RFF generally provided the best trade-off between accuracy and efficiency, often approaching the performance of the exact kernel models when using a sufficiently large number of random features. RB also improved over the linear baseline, although with less stable behavior across different settings. In particular, on the synthetic dataset, RFF consis-

tently outperformed RB. This behavior is likely due to the smoother approximation provided by Random Fourier Features, which are particularly well suited for approximating continuous kernels such as the RBF kernel. Moreover, the RBF kernel tends to be more robust to noise and small local perturbations in the data, an important aspect in the synthetic dataset where label noise is explicitly introduced. As a consequence, Random Fourier Features are able to provide a smoother and more accurate approximation of the underlying decision boundary, resulting in better generalization performance compared to Random Binning Features. In contrast, Random Binning Features generate sparse and piecewise-constant partitions of the input space, leading to a less accurate approximation on complex and noisy non-linear decision boundaries. The experiments also suggest that the more non-linear the data distribution is, the larger the number of random features D required to closely approximate the behavior of the exact kernel SVM.

An important observation emerges from the experiments on the real dataset, which appears to be more linearly separable than the synthetic one. In this case, the standard linear Perceptron often outperformed the approximate kernel approaches based on random features, showing that introducing nonlinear feature mappings is not always beneficial when the data already admits a mostly linear separation.

Overall, the experiments highlight how approximate kernel methods can provide scalable and effective alternatives to exact kernel models, while also emphasizing that their advantages strongly depend on the structure and separability of the dataset.